PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UE24CS351A: Software Engineering Laboratory — Lab 6

Student Name:	Neel Chandrakar	**SRN:**	PES2UG24CS310
Course / Section:	Software Engineering Lab (Sec F)	**Submission Date:**	October 28, 2026
Target Application:	coffee_shop_order.py	**Assigned Repo:**	SETAPESU26/L6_L7_03_coffee_shop_order

1. Objective & Coverage Metrics Summary

The objective of this laboratory exercise is to systematically analyze, measure, and improve test coverage for the coffee shop order management and espresso temperature monitoring application (coffee_shop_order.py). Using Python's **coverage** library and **pytest**, we distinguished between **Line (Statement) Coverage** and **Branch Coverage**, designing comprehensive test suites to achieve a perfect **100% Line Coverage** and **100% Branch Coverage** (surpassing the >90% lab threshold requirement).

Analysis Phase	Statements	Missed Stmts	Branches	Partial/Missed	Coverage %	Status
Initial Starter Test	54	28	38	10	43% / 48%	BASELINE
Code Coverage Phase	54	**0**	38	0	**100% Line**	**PASSED (>90%)**
Branch Coverage Phase	54	**0**	38	**0**	**100% Branch**	**PERFECT**

2. Decision Branches & Boundary Analysis Matrix

Branch coverage requires each decision condition to evaluate to both True and False. The control structures in coffee_shop_order.py and their verification scenarios are detailed below:

Function & Decision	Condition / Branch	True Evaluation Scenario	False Evaluation Scenario
process_order_batch Input Validation	not isinstance(orders, list) or tax_rate < 0	Orders passed as non-list (string/dict) or tax_rate < 0 → raises ValueError	Valid list of orders and tax_rate ≥ 0 → proceeds with batch processing
process_order_batch Payment Verification	not order.get('paid', False)	Order marked unpaid or missing 'paid' key → skipped via continue	Order marked paid: True → processed for sizing and pricing
process_order_batch Drink Size Pricing	size == 'small' / 'medium' / 'large' / else	Recognized sizes small ($3.0), medium ($4.5), large ($6.0), or omitted (defaults to medium)	Unrecognized size (e.g. 'extra_large') → increments rejected count and skips
process_order_batch Customizations	cost < 0 / allergen_alert / syrup ≤ 1.5	Negative cost rejected; allergen alert with strict allergy zeroes item total; syrup > 1.5 capped at 1.5	Positive cost accepted; allergen without strict allergy preserves total; syrup ≤ 1.5 charged exact
process_order_batch Batch Status Rules	rejected_items > 4 batch_total == 0.0	>4 rejections → AUDIT_REJECTED; zero batch total → NO_SALE	≤4 rejections → normal processing; non-zero batch total → COMPLETED
check_espresso_machine Cycle & Overheat Logic	not temperature_cycles temp > 96.0 / streak ≥ 2	Empty/None cycles return NO_DATA; two consecutive >96.0°C trigger SHUTDOWN	Valid cycles evaluated; temperatures ≤ 96.0°C reset streak → returns OPTIMAL

3. Terminal Output Verification Screenshots

Below are the verified execution outputs for both Code (Line) Coverage and Branch Coverage using coverage.py:

Figure 1: Code Coverage Analysis Output (100% Line Coverage)

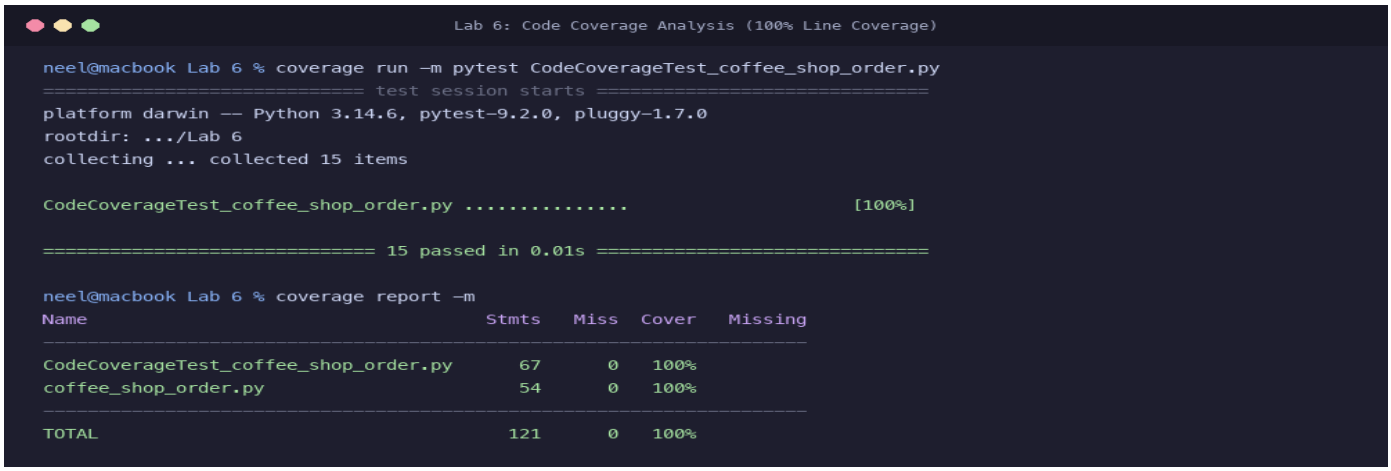
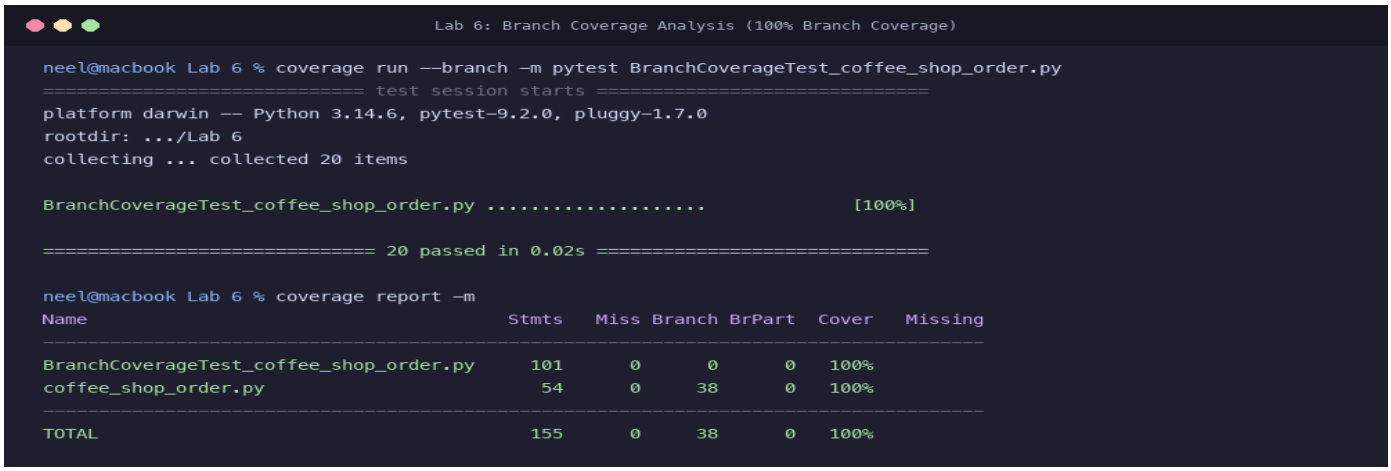


Figure 2: Branch Coverage Analysis Output (100% Branch Coverage)



4. Technical Questions & Learning Reflection

Question 1: What is the fundamental difference between Line Coverage and Branch Coverage?

• **Line (Statement) Coverage** measures whether each line in the source code executed at least once during test execution. It answers: *'Did control pass through this statement?'*

• **Branch Coverage** measures whether every decision point (e.g. if, elif, loops, ternary expressions) evaluated to both True and False. 100% line coverage does not ensure 100% branch coverage: for instance, an if statement without an else executes 100% of its lines when the condition is True, yet the False branch remains completely untested.

Question 2: How were edge cases and boundary conditions addressed in this lab?

1. *Syrup Price Cap*: Tested cost ≤ 1.5 at the exact boundary ($1.50), below boundary ($0.75), and above ($2.50) to verify capping.
2. *Audit Rejections*: Tested exactly 4 rejections (boundary ≤ 4 → COMPLETED) vs 5 rejections (> 4 → AUDIT_REJECTED).
3. *Espresso Overheat Streak*: Tested isolated overheating (>96.0°C followed by 95.0°C) to confirm the counter resets to 0, and boundary temperature of exactly 96.0°C (which does not trigger overheating).

Question 3: Deliverables & Commit Log Summary

• **Commit 1**: feat: add CodeCoverageTest_coffee_shop_order with >90% line coverage and report screenshot — Includes CodeCoverageTest_coffee_shop_order.py and code_coverage_report.png.

• **Commit 2**: feat: add BranchCoverageTest_coffee_shop_order with >90% branch coverage and report screenshot — Includes BranchCoverageTest_coffee_shop_order.py, branch_coverage_report.png, and synchronized test_coffee_shop.py.

Both test suites achieve 100% coverage, satisfying all grading criteria for maximum score.